

Helm + Argo CD + ApplicationSet

Multi-Environment GitOps Practical Runbook

This guide follows the user's current AKS lab: one reusable Helm chart, separate environment values, and an Argo CD ApplicationSet Git directory generator for dev, QA, and prod.

1. Target Repository Structure

```
Helm/
├── argocd/
│   ├── applications/
│   │   └── app-dev.yaml
│   └── charts/
│       ├── frontend/
│       │   ├── Chart.yaml
│       │   └── templates/
│       │       ├── deployment.yaml
│       │       ├── service.yaml
│       │       ├── ingress.yaml
│       │       ├── hpa.yaml
│       │       ├── serviceaccount.yaml
│       │       └── ...
│       └── environment/
│           ├── dev/
│           │   ├── frontend/
│           │   └── values.yaml
│           ├── qa/
│           │   ├── frontend/
│           │   └── values.yaml
│           └── prod/
│               ├── frontend/
│               └── values.yaml
```

The chart-level charts/frontend/values.yaml has been removed. Environment-specific values.yaml files are the values source.

2. Architecture

```
Git repository
|
+-- charts/frontend          <-- reusable Helm templates
|
+-- environment/dev/...     <-- dev values
+-- environment/qa/...      <-- QA values
+-- environment/prod/...    <-- prod values
|
v
ApplicationSet
|
+--> frontend-dev --> Helm chart + dev values --> frontend-dev namespace
+--> frontend-qa  --> Helm chart + QA values  --> frontend-qa namespace
+--> frontend-prod --> Helm chart + prod values --> frontend-prod namespace
```

3. Current Problem: ApplicationSet CRD

The ApplicationSet controller was running Argo CD v3.5.3, but the ApplicationSet CRD was missing. The controller therefore entered CrashLoopBackOff with 'no matches for kind ApplicationSet'. After the cluster connection recovered, a normal client-side apply produced an annotation-size error. Use server-side apply against the locally downloaded CRD.

4. Step-by-Step Commands

Check current Kubernetes context

```
kubectl config current-context
```

Check cluster

```
kubectl cluster-info
```

Check nodes

```
kubectl get nodes
```

Check Argo CD pods

```
kubectl get pods -n argocd
```

Check ApplicationSet controller

```
kubectl get deployment argocd-applicationset-controller -n argocd
```

Check controller pod

```
kubectl get pods -n argocd | Select-String applicationset
```

Check CRD

```
kubectl get crd applicationsets.argoproj.io
```

Go to ApplicationSet directory

```
cd E:\argocd\helm-lab\Helm\argocd\applications
```

Download matching v3.5.3 ApplicationSet CRD

```
Invoke-WebRequest `
  -Uri "https://raw.githubusercontent.com/argoproj/argo-cd/v3.5.3/manifests/crds/applicationset-crd.yaml" `
  -OutFile ".\applicationset-crd.yaml"
```

Install CRD with server-side apply

```
kubectl apply --server-side --force-conflicts -f .\applicationset-crd.yaml
```

Verify CRD

```
kubectl get crd applicationsets.argoproj.io
```

Verify API resource

```
kubectl api-resources | Select-String ApplicationSet
```

Check controller

```
kubectl get pods -n argocd | Select-String applicationset
```

Check controller logs

```
kubectl logs deployment/argocd-applicationset-controller -n argocd --tail=30
```

5. Helm Validation

DEV lint

```
helm lint .\charts\frontend -f .\environment\dev\frontend\values.yaml
```

DEV render test

```
helm template frontend-dev .\charts\frontend `
-f .\environment\dev\frontend\values.yaml
```

QA lint

```
helm lint .\charts\frontend -f .\environment\qa\frontend\values.yaml
```

QA render test

```
helm template frontend-qa .\charts\frontend `
-f .\environment\qa\frontend\values.yaml
```

PROD lint

```
helm lint .\charts\frontend -f .\environment\prod\frontend\values.yaml
```

PROD render test

```
helm template frontend-prod .\charts\frontend `
-f .\environment\prod\frontend\values.yaml
```

6. ApplicationSet YAML

```
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet

metadata:
  name: frontend-environments
  namespace: argocd

spec:
  generators:
    - git:
        repoURL: https://github.com/ranagit-ux/Helm.git
        revision: main
        directories:
          - path: environment/*/frontend

  template:
    metadata:
      name: 'frontend-{{path[1]}}'

    spec:
      project: default

      sources:
        - repoURL: https://github.com/ranagit-ux/Helm.git
          targetRevision: main
          path: charts/frontend
          helm:
            valueFiles:
              - '$values/{{path}}/values.yaml'

        - repoURL: https://github.com/ranagit-ux/Helm.git
          targetRevision: main
          ref: values

      destination:
        server: https://kubernetes.default.svc
        namespace: 'frontend-{{path[1]}}'

      syncPolicy:
        automated:
          selfHeal: true
          prune: true
        syncOptions:
          - CreateNamespace=true
```

7. Git Commit and Push

```
cd E:\argocd\helm-lab\Helm
git status
git add .
git commit -m "Configure Helm multi environment ApplicationSet"
git push origin main
```

8. Apply and Verify ApplicationSet

```
kubectl apply -f .\argocd\applications\app-dev.yaml
kubectl get applicationset -n argocd
kubectl describe applicationset frontend-environments -n argocd
kubectl get applications -n argocd
kubectl get applications -n argocd -o wide
```

Expected generated Applications

```
frontend-dev
frontend-qa
frontend-prod
```

9. Verify Namespaces and Workloads

```
kubectl get namespaces

kubectl get all -n frontend-dev
kubectl get all -n frontend-qa
kubectl get all -n frontend-prod

kubectl get pods -n frontend-dev
kubectl get pods -n frontend-qa
kubectl get pods -n frontend-prod
```

10. Daily Automation Flow

```
Change environment/dev/frontend/values.yaml
|
v
git add .
git commit -m "Update dev frontend"
git push origin main
|
v
Argo CD detects Git change
|
v
ApplicationSet maintains frontend-dev
|
v
Helm renders charts/frontend + dev values
|
v
Kubernetes updates frontend-dev namespace
```

11. Quick Troubleshooting

ApplicationSet kind not found: Check the CRD with `kubectl get crd applicationsets.argoproj.io`.

CRD annotation too long: Use server-side apply: `kubectl apply --server-side --force-conflicts -f .\applicationset-crd.yaml`.

http2 client connection lost: Test `kubectl cluster-info` and `kubectl get nodes` before retrying the CRD.

Controller CrashLoopBackOff: Check `kubectl logs deployment/argocd-applicationset-controller -n argocd --previous`.

Helm values errors: Run `helm lint` and `helm template` separately for dev, QA, and prod.